

LAMPIRAN A - DOKUMENTASI KODE SUMBER (BAGIAN 1)

Sistem Solar Dryer IoT - Source Code Lengkap

1. FIRMWARE ESP32

File: esp32/smart_ac.ino

Deskripsi: Firmware utama ESP32 untuk membaca 10 sensor DHT22, mengontrol blower BTS7960 via PWM, logging ke SD Card, dan sinkronisasi data ke Firebase Realtime Database.

```
* SOLAR DRYER IOT - ESP32 FIRMWARE (AUTO + MANUAL VERSION + NTP RTC CALIBRATION)
* =====
* This system monitors 5 DHT22 sensors & 5 DS18B20 sensors, logs data to SD card,
* and syncs with Firebase RTDB. Blower supports AUTO and MANUAL control via Firebase.
* RTC automatically calibrates via Internet (NTP) on startup.
* * HARDWARE WIRING:
* -----
* Component      | ESP32 Pin | Details
* -----|-----|-----
* DHT22 (1-5)     | 32, 33, 25, 27, 26
* DS18B20 (6-10)  | 14, 17, 13, 16, 4  <-- PASTIKAN PAKAI RESISTOR PULL-UP 4.7K
* Blower 1 (RPWM) | 2          | Connect to RPWM on BTS7960
* Blower 1 (Enable) | 15         | (DIPINDAH DARI PIN 4) R_EN & L_EN on BTS7960
* Blower 1 (LPWM) | GND        | Connect to ESP32 GND
* SD Card CS      | 5          | SPI CS
* SD Card SCK     | 18         | SPI SCK
```

```

* SD Card MISO      | 19      | SPI MISO
* SD Card MOSI      | 23      | SPI MOSI
* RTC SDA           | 21      | I2C SDA
* RTC SCL           | 22      | I2C SCL
* -----
*/

#include <WiFi.h>
#include "time.h"          // LIBRARY TAMBAHAN UNTUK NTP WAKTU INTERNET
#include <Firebase_ESP_Client.h>
#include <DHT.h>
#include <Wire.h>
#include <RTCLib.h>
#include <SPI.h>
#include <SD.h>
#include <OneWire.h>       // LIBRARY TAMBAHAN UNTUK DS18B20
#include <DallasTemperature.h> // LIBRARY TAMBAHAN UNTUK DS18B20
#include <addons/TokenHelper.h>
#include <addons/RTDBHelper.h>

// ===== KONFIGURASI =====

// WiFi Credentials
#define WIFI_SSID      "POCO X3 NFC"
#define WIFI_PASSWORD "abcd1234"

// Define Firebase credentials
#define API_KEY "AIzaSyA-T8JuoAYfUoBVJvu0Vt-I3WSYgJsgHME"
#define DATABASE_URL "https://solar-air-dryer-iot-f797f-default-rtdb.asia-southeast1.firebaseio.com/"
#define USER_EMAIL "abel@rizky.com"
#define USER_PASSWORD "Iot123"

#define JUMLAH_SENSOR 10

// === DHT22 Sensor (5 unit pertama) ===
#define DHTTYPE DHT22
#define JUMLAH_DHT 5
int dhtPins[JUMLAH_DHT] = {32, 33, 25, 27, 26};
DHT dhtSensors[JUMLAH_DHT] = {

```

```

DHT(32, DHTTYPE), DHT(33, DHTTYPE), DHT(25, DHTTYPE),
DHT(27, DHTTYPE), DHT(26, DHTTYPE)
};

// === DS18B20 Sensor (5 unit terakhir) ===
#define JUMLAH_DS 5
int dsPins[JUMLAH_DS] = {14, 17, 13, 16, 4};
OneWire oneWire[JUMLAH_DS] = { OneWire(14), OneWire(17), OneWire(13), OneWire(16), OneWire(4) };
DallasTemperature dsSensors[JUMLAH_DS]; // Di-inisialisasi di setup()

// === Kipas / Blower (BTS7960) ===
#define JUMLAH_KIPAS 1
int relayPins[JUMLAH_KIPAS] = {2}; // Pin RPWM (Sinyal PWM Speed)
int enablePins[JUMLAH_KIPAS] = {15}; // DIPINDAH KE PIN 15 AGAR TIDAK BENTROK DENGAN PIN 4 (DS18B20)

// === SD Card ===
#define SD_CS_PIN 5

// === Interval (ms) ===
#define SENSOR_INTERVAL 60000
#define HISTORY_INTERVAL 60000

// ===== VARIABEL GLOBAL =====

RTC_DS3231 rtc;
bool sdReady = false;

FirebaseData fbdo;
FirebaseData streamFans;
FirebaseAuth fbAuth;
FirebaseConfig fbConfig;

float sensorTemp[JUMLAH_SENSOR];
float sensorHum[JUMLAH_SENSOR];
float avgTemp = 0;
float avgHum = 0;

int kipasSpeed[JUMLAH_KIPAS] = {0};
String controlMode = "auto"; // Mode kontrol: "auto" atau "manual"

```

```

unsigned long lastSensorRead = 0;
unsigned long lastHistoryPush = 0;
unsigned long lastWiFiCheck = 0;
int lastLoggedMinute = -1;

// ===== HELPER =====

String getHumidityCategory(float h) {
    if (isnan(h)) return "N/A";
    if (h < 30) return "Rendah";
    else if (h <= 60) return "Sedang";
    return "Tinggi";
}

void setBlowerSpeed(int idx, int speedPct) {
    if (idx < 0 || idx >= JUMLAH_KIPAS) return;
    if (speedPct < 0) speedPct = 0;
    if (speedPct > 100) speedPct = 100;

    kipasSpeed[idx] = speedPct;
    if (relayPins[idx] >= 0) {
        int pwmValue = map(speedPct, 0, 100, 0, 255);

        // Logika khusus BTS7960: Nyalakan Enable pin saat kecepatan > 0
        if (speedPct > 0) {
            digitalWrite(enablePins[idx], HIGH); // Aktifkan driver
            analogWrite(relayPins[idx], pwmValue); // Kirim sinyal PWM
        } else {
            analogWrite(relayPins[idx], 0); // Hentikan PWM
            digitalWrite(enablePins[idx], LOW); // Matikan driver (hemat daya)
        }
    }

    Serial.printf(" Kipas %d → %d%%\n", idx + 1, speedPct);
}

// ===== AUTO CONTROL =====

void autoControlBlower() {

```

```

if (avgTemp == 0 && avgHum == 0) return;

int targetSpeed = 0;

if (avgTemp >= 55.0) {
    targetSpeed = 100;
} else if (avgTemp >= 49.0) {
    targetSpeed = 75;
} else if (avgTemp >= 42.0) {
    targetSpeed = 50;
} else if (avgTemp >= 35.0) {
    targetSpeed = 25;
} else {
    targetSpeed = 0;
}

if (kipasSpeed[0] != targetSpeed) {
    Serial.printf(" [AUTO] Suhu: %.1f°C → Blower diubah ke %d%%\n", avgTemp, targetSpeed);
    setBlowerSpeed(0, targetSpeed);

    if (Firebase.ready()) {
        Firebase.RTDB.setInt(&fbdo, "/blowers/blower1/speed", targetSpeed);
    }
}

// ===== WIFI =====

void connectWiFi() {
    Serial.printf("Connecting to WiFi: %s", WIFI_SSID);
    WiFi.begin(WIFI_SSID, WIFI_PASSWORD);

    unsigned long startAttemptTime = millis();

    while (WiFi.status() != WL_CONNECTED && millis() - startAttemptTime < 20000) {
        Serial.print(".");
        delay(500);
    }
}

```

```

    if (WiFi.status() != WL_CONNECTED) {
        Serial.println("\n❌ Connection Failed! System will continue and retry in loop.");
    } else {
        Serial.printf("\n✅ Connected! IP: %s\n", WiFi.localIP().toString().c_str());
    }
}

// ===== NTP SYNC WAKTU =====

void syncNTPtoRTC() {
    const char* ntpServer = "pool.ntp.org";
    const long  gmtOffset_sec = 25200; // Offset WIB = 7 jam * 3600
    const int   daylightOffset_sec = 0;

    configTime(gmtOffset_sec, daylightOffset_sec, ntpServer);
    Serial.println("Mengambil waktu dari NTP Server...");

    struct tm timeinfo;
    if (getLocalTime(&timeinfo, 10000)) { // Tunggu maksimal 10 detik
        // Update RTC dengan waktu dari Internet
        rtc.adjust(DateTime(timeinfo.tm_year + 1900, timeinfo.tm_mon + 1, timeinfo.tm_mday,
                            timeinfo.tm_hour, timeinfo.tm_min, timeinfo.tm_sec));
        Serial.println("✅ RTC berhasil dikalibrasi via Internet (NTP)!");
    } else {
        Serial.println("❌ Gagal sinkronisasi NTP, menggunakan waktu RTC yang ada.");
    }
}

// ===== FIREBASE =====

void fanStreamCallback(FirebaseStream data) {
    if (controlMode != "manual") return;
    if (data.dataType() != "int" && data.dataType() != "double" && data.dataType() != "float") return;

    String path = data.dataPath();
    for (int i = 0; i < JUMLAH_KIPAS; i++) {
        if (path == "/blower" + String(i + 1) + "/speed") {
            setBlowerSpeed(i, data.intData());
            break;
        }
    }
}

```

```

    }
}

void controlModeCallback(FirebaseStream data) {
    if (data.dataType() != "string") return;
    String newMode = data.stringData();
    if (newMode == "auto" || newMode == "manual") {
        controlMode = newMode;
        Serial.printf(" [MODE] Kontrol diubah ke: %s\n", controlMode.c_str());

        if (controlMode == "auto") {
            autoControlBlower();
        }
    }
}

void controlModeTimeoutCallback(bool timeout) {
    if (timeout) Serial.println("Control mode stream timeout, resuming...");
}

void streamTimeoutCallback(bool timeout) {
    if (timeout) Serial.println("Stream timeout, resuming...");
}

void initFirebase() {
    fbConfig.api_key      = API_KEY;
    fbConfig.database_url = DATABASE_URL;
    fbAuth.user.email     = USER_EMAIL;
    fbAuth.user.password  = USER_PASSWORD;
    fbConfig.token_status_callback = tokenStatusCallback;

    Firebase.begin(&fbConfig, &fbAuth);
    Firebase.reconnectNetwork(true);

    fbdo.setBSSLBufferSize(1024, 1024);
    fbdo.setResponseSize(1024);
    streamFans.setBSSLBufferSize(2048, 1024);
    streamFans.setResponseSize(1024);
}

```

```

Serial.println("Menunggu autentikasi Firebase...");
int timeout = 0;
while (!Firebase.ready() && timeout < 50) {
    delay(100);
    timeout++;
}

if (Firebase.ready()) {
    Serial.println("Firebase terhubung!");
} else {
    Serial.println("Firebase Auth Timeout (menunggu WiFi terhubung di background...)");
}

for (int i = 0; i < JUMLAH_KIPAS; i++) {
    String key = "/blowers/blower" + String(i + 1);
    Firebase.RTDB.setInt(&fbdo, key + "/speed", 0);
    Firebase.RTDB.setString(&fbdo, key + "/name", "Blower " + String(i + 1));
}

Firebase.RTDB.setString(&fbdo, "/control_mode", "auto");
controlMode = "auto";

Firebase.RTDB.beginStream(&streamFans, "/blowers");
Firebase.RTDB.setStreamCallback(&streamFans, fanStreamCallback, streamTimeoutCallback);

static FirebaseData streamMode;
streamMode.setBSSLBufferSize(1024, 512);
Firebase.RTDB.beginStream(&streamMode, "/control_mode");
Firebase.RTDB.setStreamCallback(&streamMode, controlModeCallback, controlModeTimeoutCallback);
}

// ===== HELPER JSON =====

void buildDetailedJson(FirebaseJson &json, DateTime now) {
    FirebaseJsonArray sensors;

    char dateBuf[11];
    sprintf(dateBuf, "%04d-%02d-%02d", now.year(), now.month(), now.day());

```

```

char timeBuf[9];
sprintf(timeBuf, "%02d:%02d:%02d", now.hour(), now.minute(), now.second());

json.set("avg_t", avgTemp);
json.set("avg_h", avgHum);
json.set("tanggal", dateBuf);
json.set("waktu", timeBuf);

for (int i = 0; i < JUMLAH_SENSOR; i++) {
    FirebaseJson s;
    s.set("sensor", i + 1);

    if (isnan(sensorTemp[i])) {
        s.set("t", "null");
    } else {
        s.set("t", sensorTemp[i]);
    }

    if (isnan(sensorHum[i])) {
        s.set("h", "null");
    } else {
        s.set("h", sensorHum[i]);
    }

    sensors.add(s);
}
json.set("sensor_data", sensors);
}

// ===== BACA SENSOR =====

void readAllSensors() {
    float totalT = 0, totalH = 0;
    int validCountT = 0;
    int validCountH = 0;

    DateTime now = rtc.now();
    Serial.printf("\n=== %04d-%02d-%02d %02d:%02d:%02d ===\n",
        now.year(), now.month(), now.day(),

```

```

now.hour(), now.minute(), now.second());

// 1. Baca 5 Sensor DHT22
for (int i = 0; i < JUMLAH_DHT; i++) {
    sensorTemp[i] = dhtSensors[i].readTemperature();
    sensorHum[i] = dhtSensors[i].readHumidity();

    if (!isnan(sensorTemp[i]) && !isnan(sensorHum[i])) {
        totalT += sensorTemp[i];
        totalH += sensorHum[i];
        validCountT++;
        validCountH++;
        Serial.printf(" Sensor %2d (DHT): %.1f°C | %.1f%% (%s)\n",
            i + 1, sensorTemp[i], sensorHum[i],
            getHumidityCategory(sensorHum[i]).c_str());
    } else {
        Serial.printf(" Sensor %2d (DHT): GAGAL\n", i + 1);
    }
}

// 2. Baca 5 Sensor DS18B20
for (int i = 0; i < JUMLAH_DS; i++) {
    int idx = i + JUMLAH_DHT; // Index 5 - 9
    dsSensors[i].requestTemperatures();
    float t = dsSensors[i].getTempCByIndex(0);

    if (t != DEVICE_DISCONNECTED_C && t != 85.0) { // 85.0 adalah kode error default DS18B20
        sensorTemp[idx] = t;
        sensorHum[idx] = NAN; // DS18B20 tidak punya sensor kelembaban

        totalT += t;
        validCountT++;

        Serial.printf(" Sensor %2d (DS18): %.1f°C | N/A (Tanpa Kelembaban)\n", idx + 1, t);
    } else {
        sensorTemp[idx] = NAN;
        sensorHum[idx] = NAN;
        Serial.printf(" Sensor %2d (DS18): GAGAL\n", idx + 1);
    }
}

```

```

}

// Hitung Rata-rata
if (validCountT > 0) avgTemp = totalT / validCountT; else avgTemp = 0;
if (validCountH > 0) avgHum = totalH / validCountH; else avgHum = 0;

if (validCountT > 0 || validCountH > 0) {
    Serial.printf(" Rata-rata : %.1f°C | %.1f%% (Suhu: %d valid, Hum: %d valid)\n",
        avgTemp, avgHum, validCountT, validCountH);

    if (controlMode == "auto") {
        autoControlBlower();
    }
}

// Update ke Firebase
if (Firebase.ready()) {
    FirebaseJson detailedData;
    buildDetailedJson(detailedData, now);
    Firebase.RTDB.setJSON(&fbdo, "/sensor_data", &detailedData);
}

// Update SD Card
if (sdReady && now.minute() != lastLoggedMinute) {
    lastLoggedMinute = now.minute();
    logToSD(now);
}

}

// ===== SD CARD LOGGING =====

void logToSD(DateTime now) {
    String logLine = String(now.year()) + "-" +
        (now.month() < 10 ? "0" : "") + String(now.month()) + "-" +
        (now.day() < 10 ? "0" : "") + String(now.day()) + "," +
        (now.hour() < 10 ? "0" : "") + String(now.hour()) + ":" +
        (now.minute() < 10 ? "0" : "") + String(now.minute()) + ":" +
        (now.second() < 10 ? "0" : "") + String(now.second());
}

```

```

for (int i = 0; i < JUMLAH_SENSOR; i++) {
    // Log Suhu
    if (!isnan(sensorTemp[i])) {
        logLine += "," + String(sensorTemp[i], 1);
    } else {
        logLine += ",NAN";
    }

    // Log Kelembaban
    if (!isnan(sensorHum[i])) {
        logLine += "," + String(sensorHum[i], 1);
    } else {
        logLine += ",NAN";
    }
}

File dataFile = SD.open("/data.csv", FILE_APPEND);
if (dataFile) {
    dataFile.println(logLine);
    dataFile.close();
    Serial.println(" SD: Data disimpan");
} else {
    Serial.println(" SD: Gagal menyimpan!");
}
}

// ===== FIREBASE HISTORY =====

void pushHistory() {
    if (avgTemp == 0 && avgHum == 0) return;
    if (!Firebase.ready()) return;

    DateTime now = rtc.now();
    FirebaseJson entry;
    buildDetailedJson(entry, now);
    entry.set("timestamp/.sv", "timestamp");

    if (Firebase.RTDB.pushJSON(&fbdo, "/history", &entry)) {
        Serial.println(" Firebase: History pushed (detailed)");
    }
}

```

```

    }
}

// ===== SETUP =====

void setup() {
    Serial.begin(115200);
    delay(2000);
    Serial.println("\n=====");
    Serial.println("  Solar Dryer IoT - AUTO + MANUAL");
    Serial.println("=====\\n");

    // --- Init DHT22 Sensor ---
    Serial.println("[1/6] Init 5 sensor DHT22...");
    for (int i = 0; i < JUMLAH_DHT; i++) {
        dhtSensors[i].begin();
    }
    Serial.println("      OK");

    // --- Init DS18B20 Sensor ---
    Serial.println("[2/6] Init 5 sensor DS18B20...");
    for (int i = 0; i < JUMLAH_DS; i++) {
        dsSensors[i].setOneWire(&oneWire[i]);
        dsSensors[i].begin();
    }
    Serial.println("      OK");

    // --- Init Kipas (BTS7960) ---
    Serial.println("[3/6] Init kipas (BTS7960)...");
    for (int i = 0; i < JUMLAH_KIPAS; i++) {
        if (relayPins[i] >= 0) {
            pinMode(relayPins[i], OUTPUT);
            pinMode(enablePins[i], OUTPUT);

            digitalWrite(enablePins[i], LOW);
            analogWrite(relayPins[i], 0);

            Serial.printf("      Kipas %d → PWM: GPIO %d, EN: GPIO %d\\n", i + 1, relayPins[i], enablePins[i]);
        }
    }
}

```

```

}

// --- Init RTC DS3231 ---
Serial.println("[4/6] Init RTC DS3231...");
Wire.begin(21, 22);
if (!rtc.begin()) {
    Serial.println("      GAGAL! RTC tidak terdeteksi.");
    while (1);
}

if (rtc.lostPower()) {
    Serial.println("      RTC kehilangan daya, waktu tidak valid sampai NTP sinkron.");
}

DateTime now = rtc.now();
Serial.printf("      Waktu Awal RTC → %04d-%02d-%02d %02d:%02d:%02d\n",
    now.year(), now.month(), now.day(),
    now.hour(), now.minute(), now.second());

// KODE INI DIMATIKAN AGAR TIDAK MENGGANGGU SINKRONISASI NTP
// struct timeval tv;
// tv.tv_sec = now.unixtime();
// tv.tv_usec = 0;
// settimeofday(&tv, NULL);

// --- Init SD Card ---
Serial.println("[5/6] Init SD Card...");
if (SD.begin(SD_CS_PIN)) {
    sdReady = true;
    if (!SD.exists("/data.csv")) {
        File f = SD.open("/data.csv", FILE_WRITE);
        if (f) {
            String header = "Tanggal,Waktu";
            for (int i = 1; i <= JUMLAH_SENSOR; i++) {
                header += ",T" + String(i) + ",H" + String(i);
            }
            f.println(header);
            f.close();
            Serial.println("      Header CSV dibuat");
        }
    }
}

```

```

    }
}
Serial.println("      OK");
} else {
    Serial.println("      GAGAL! Lanjut tanpa SD logging.");
}

// --- WiFi + NTP + Firebase ---
Serial.println("[6/6] Koneksi WiFi, NTP & Firebase...");
connectWiFi();

// Sinkronisasi waktu otomatis setelah WiFi terhubung
if(WiFi.status() == WL_CONNECTED) {
    syncNTPtoRTC();
}

initFirebase();

Serial.println("\n✅ Sistem siap! Blower di mode AUTO / MANUAL.\n");
}

// ===== LOOP =====

void loop() {
    unsigned long now = millis();

    // Baca sensor setiap 1 menit
    if (now - lastSensorRead >= SENSOR_INTERVAL) {
        lastSensorRead = now;
        readAllSensors();
    }

    // Push history ke Firebase setiap 1 menit
    if (now - lastHistoryPush >= HISTORY_INTERVAL) {
        lastHistoryPush = now;
        pushHistory();
    }

    // Reconnect WiFi otomatis jika terputus

```

```

    if (WiFi.status() != WL_CONNECTED) {
      if (now - lastWiFiCheck >= 15000) {
        lastWiFiCheck = now;
        Serial.println(" ⌚ WiFi terputus, menunggu auto-reconnect dari ESP32...");
      }
    }
  }
}
\n```

```

2. REACT WEB DASHBOARD

File: `react-dashboard/src/firebase.js`

****Deskripsi**:** Konfigurasi dan inisialisasi Firebase SDK untuk web.

```javascript

```

import { initializeApp, getApps, getApp } from "firebase/app";
import { getDatabase } from "firebase/database";
import { getAuth } from "firebase/auth";

```

```

const firebaseConfig = {
 apiKey: "AIzaSyA-T8JuoAYfUoBVJvu0Vt-I3WSYgJsgHME",
 authDomain: "solar-air-dryer-iot-f797f.firebaseio.com",
 databaseURL: "https://solar-air-dryer-iot-f797f-default-rtdb.asia-southeast1.firebaseio.com",
 projectId: "solar-air-dryer-iot-f797f",
 storageBucket: "solar-air-dryer-iot-f797f.firebaseio.com",
 messagingSenderId: "540412890134",
 appId: "1:540412890134:web:9bbc1e61cc2837ad5fb937",
 measurementId: "G-6ZF19TNH9H"
};

```

```

let database = null;
let auth = null;
try {
 const app = getApps().length > 0 ? getApp() : initializeApp(firebaseConfig);
 database = getDatabase(app);
 auth = getAuth(app);
} catch (e) {

```

```
 console.warn("Firebase initialization error:", e.message);
 }

export { database, auth };
```

---

## File: [react-dashboard/src/index.js](#)

---

**Deskripsi:** Entry point aplikasi React, membungkus App dengan AuthProvider.

```
import React from "react";
import ReactDOM from "react-dom/client";
import App from "./App";
import { AuthProvider } from "../contexts/AuthContext";

const root = ReactDOM.createRoot(document.getElementById("root"));
root.render(
 <React.StrictMode>
 <AuthProvider>
 <App />
 </AuthProvider>
 </React.StrictMode>
);
```

---

## File: [react-dashboard/src/parseCsv.js](#)

---

**Deskripsi:** Utilitas untuk parsing data CSV dari SD Card menjadi format yang dapat digunakan oleh chart dan tabel.

```
const SENSOR_COUNT = 10;

export function parseDummyCsv(text) {
 const lines = text.trim().split("\n");
```

```

const rows = lines.slice(1);
const history = [];
for (const row of rows) {
 const cols = row.split(",");
 if (cols.length < 22) continue;
 const rawDate = cols[0].trim();
 const rawTime = cols[1].trim();
 const pad = (v) => String(v).padStart(2, "0");
 let isoDate;
 if (rawDate.includes("/")) {
 const dp = rawDate.split("/");
 isoDate = `${dp[2]}-${pad(dp[0])}-${pad(dp[1])}`;
 } else {
 const dp = rawDate.split("-");
 isoDate = `${dp[0]}-${pad(dp[1])}-${pad(dp[2])}`;
 }
 const timeParts = rawTime.split(":");
 const isoTime = `${pad(timeParts[0])}:${pad(timeParts[1])}:${pad(timeParts[2] || 0)}`;
 const ts = new Date(`${isoDate}T${isoTime}`).getTime();
 if (isNaN(ts)) continue;
 let tSum = 0, hSum = 0, tCount = 0, hCount = 0;
 const sensors = [];
 for (let i = 0; i < SENSOR_COUNT; i++) {
 const tRaw = cols[2 + i * 2]?.trim().toUpperCase();
 const hRaw = cols[3 + i * 2]?.trim().toUpperCase();
 const tVal = (tRaw && tRaw !== "NAN") ? parseFloat(tRaw) : NaN;
 const hVal = (hRaw && hRaw !== "NAN") ? parseFloat(hRaw) : NaN;
 const tOk = !isNaN(tVal) && tVal > 0;
 const hOk = !isNaN(hVal) && hVal > 0;
 sensors.push({ temperature: tOk ? tVal : null, humidity: hOk ? hVal : null });
 if (tOk) { tSum += tVal; tCount++; }
 if (hOk) { hSum += hVal; hCount++; }
 }
 if (tCount === 0) continue;
 history.push({
 timestamp: ts,
 temperature: +(tSum / tCount).toFixed(1),
 humidity: +(hSum / hCount).toFixed(1),
 sensors, validSensors: tCount,
 });
}

```

```
 });
 }
 return history;
}
```